

INTERNSHIP / UDP / TRAINING REPORT

(Monthly Progress Report)

1. Student Information

Student Name	AAKASH BHARATBHAI JAYANI
Enrollment Number	23SE02IT140
Program/Branch	B.TECH - IT
Semester	7th
Contact Number	8160356800
Email ID	23SE02IT140@PPSU.AC.IN

2. Internship / UDP / Training Details

Type	UDP
Company/Organization	PPSU
Department/Domain	SOE
Mentor Name (Company)	
Mentor Name (Institute)	Mr. Anurag Yadav
Start Date	
End Date	

3. Work Progress Summary

a) Project Title:

Cache-Craft: Autonomous Semantic RAG Router for LLM API Cost Optimization Using Dense Vector Embeddings

b) Problem Statement:

Large Language Model (LLM) APIs such as OpenAI GPT-4, Google Gemini, and Anthropic Claude have become foundational components in modern intelligent applications — from customer support chatbots and internal knowledge assistants to code generation tools and automated report writers. However, every API call incurs a direct monetary cost (typically \$0.10–\$10.00 per million tokens) and introduces latency of 2–5 seconds per request. In production environments handling thousands of queries daily, a significant proportion of incoming queries are semantically identical or highly similar to previously answered questions: a user asking “What was our Q3 revenue?” and another asking “Show me third quarter revenue” are functionally the same query, yet both trigger separate, full-cost LLM API calls. Existing caching solutions (e.g., Redis, Memcached) rely on exact string matching, which fails entirely when queries are rephrased, reworded, or contain synonyms. This project addresses this gap by building Cache-Craft — an autonomous semantic caching middleware that uses dense vector embeddings and approximate nearest neighbor (ANN) search to identify semantically equivalent queries and serve cached responses instantly, reducing LLM API costs by up to 60–70% while delivering sub-50ms response times on cache hits.

c) Objectives:

- Design and implement a semantic caching middleware layer (Cache-Craft) that intercepts LLM API requests and checks for semantically similar previously answered queries before forwarding to the cloud LLM.
- Integrate a locally-hosted dense embedding model (all-MiniLM-L6-v2, 384 dimensions) running on GPU to convert natural language queries into high-dimensional vector representations in under 5 milliseconds.
- Implement vector similarity search using PostgreSQL with the pgvector extension and HNSW (Hierarchical Navigable Small World) indexing for sub-15ms approximate nearest neighbor lookup across 100,000+ cached entries.
- Build a configurable similarity threshold system (default: 0.85 cosine similarity) that allows users to tune the trade-off between cache hit rate and response accuracy via a web-based dashboard.
- Develop an optional Redis-based L1 hot cache layer for exact or near-exact query matches, achieving sub-1ms retrieval for the most frequently repeated queries.
- Integrate with the Google Gemini 2.0 Flash API as the upstream cloud LLM, with a modular architecture allowing easy substitution of any LLM provider (OpenAI, Anthropic, Ollama).
- Build a real-time analytics dashboard using Next.js and Tailwind CSS that visualizes key metrics: cache hit rate, cumulative API cost savings, average response latency (cache hit vs. cache miss), and cache utilization over time.
- Conduct a rigorous quantitative evaluation using a benchmark of 1,200 queries (500 unique, 500 paraphrased, 200 unrelated) to measure precision, recall, F1 score, and cost reduction percentage at varying similarity thresholds.
- Containerize the entire application stack (FastAPI backend, PostgreSQL + pgvector, Redis, Next.js frontend) using Docker Compose for one-command reproducible deployment.

d) Proposed Methodology:***Phase 1 — System Architecture & Environment Setup:***

The project begins with designing the overall system architecture as a layered middleware. Docker Compose is configured to orchestrate four services: PostgreSQL 16 with pgvector extension, Redis 7 (Alpine) with LRU eviction policy and 256MB memory cap, FastAPI backend with NVIDIA GPU passthrough for the embedding model, and Next.js 15 frontend. Environment variables are managed via .env files for API keys, database credentials, and

configurable parameters (similarity threshold, TTL, cache size limits). The pgvector extension is initialized with the HNSW index type configured for cosine distance operations.

Phase 2 — Embedding Engine & Vector Storage:

The SentenceTransformers library is used to load the all-MiniLM-L6-v2 model onto the RTX 4060 GPU using CUDA acceleration. Each incoming query is normalized (lowercased, whitespace-trimmed) and encoded into a 384-dimensional dense vector. The vector, along with the original query text and metadata, is stored in a PostgreSQL table with a VECTOR(384) column. An HNSW index is created on this column with parameters $m=16$ and $ef_construction=64$, enabling logarithmic-time approximate nearest neighbor search even as the cache grows to 100,000+ entries.

Phase 3 — Cache Router Logic (Core Algorithm):

The Cache Router is the central intelligence of Cache-Craft. For each incoming query: (1) A SHA-256 hash is computed and checked against Redis for an exact match (L1 cache, $\sim 0.1\text{ms}$). (2) If no Redis hit, the query is embedded and a cosine similarity search is performed against pgvector (L2 cache, $\sim 5\text{--}15\text{ms}$). The query retrieves the single most similar cached entry. (3) If the similarity score meets or exceeds the configurable threshold (default 0.85), the cached response is returned as a cache hit. The entry's hit_count is incremented and last_accessed timestamp is updated. (4) If no match is found, the query is forwarded to the Gemini 2.0 Flash API. The response, along with token count and estimated cost, is stored in the cache for future reuse.

Phase 4 — Upstream LLM Integration:

The Google Gemini 2.0 Flash model is integrated via the google-generativeai Python SDK. A modular LLM client class is designed with a common interface, allowing future substitution with OpenAI, Anthropic, or local Ollama models without changing the cache router logic. Each LLM response is tagged with the model name, token usage, and estimated cost based on the provider's published pricing. The system tracks cumulative API spending to quantify savings achieved by semantic caching.

Phase 5 — Next.js Dashboard & Real-Time Analytics:

A professional-grade dashboard is built using Next.js 15 (App Router) with Tailwind CSS v4 for styling and Recharts for data visualization. The dashboard includes: (a) A chat-style query interface showing real-time cache hit/miss indicators and similarity scores, (b) An analytics overview page with KPI cards for total queries, hit rate percentage, cumulative cost saved, and average latency, (c) Time-series line charts showing cost savings and hit rate trends, (d) A cache explorer page for browsing, searching, and manually invalidating cached entries, and (e) A settings panel for adjusting the similarity threshold, TTL duration, and model selection. Server-Sent Events (SSE) are used to push live metric updates to the frontend.

Phase 6 — Benchmarking & Evaluation:

A comprehensive evaluation suite is developed using a corpus of 1,200 test queries divided into three categories: 500 unique seed queries (guaranteed cache misses to populate the cache), 500 paraphrased versions of the seed queries (expected cache hits to test semantic matching), and 200 completely unrelated queries (expected cache misses to test false positive resistance). The system is evaluated at similarity thresholds ranging from 0.70 to 0.95 in increments of 0.05. For each threshold, precision, recall, F1 score, cache hit rate, average latency, and total cost savings are recorded. A comparison study is conducted against three baselines: no cache, exact-match Redis cache, and Cache-Craft semantic cache.

e) Expected Outcomes:

- A fully functional semantic caching middleware deployable via a single Docker Compose command, capable of intercepting and routing LLM API requests through a two-tier cache (Redis L1 + pgvector L2).
- Measurable reduction in LLM API costs of 60–70% on repeated and paraphrased query workloads, validated through systematic benchmarking with 1,200 test queries.
- Sub-50ms end-to-end response time on cache hits (compared to 2,000–5,000ms for cloud LLM calls), representing a 40–100x latency improvement.
- A configurable similarity threshold system with empirical analysis showing the optimal threshold (expected: 0.85) that maximizes F1 score across precision and recall.
- A professional, dark-mode analytics dashboard with real-time visualization of cache hit rates, cumulative cost savings, and latency comparisons.
- A reusable, modular architecture with a pluggable LLM client interface supporting Gemini, OpenAI, Anthropic, and local Ollama models without code changes to the caching layer.
- A comprehensive evaluation report comparing semantic caching (Cache-Craft) against exact-match caching (Redis) and no-cache baselines, demonstrating 3–5x improvement in cache hit rates through semantic matching.

f) Future Scope:

- Implement adaptive threshold learning using reinforcement feedback — when users re-ask a question after receiving a cached response, the system detects a false positive and automatically adjusts the similarity threshold downward for that query cluster.
- Add namespace-based cache isolation to support multi-tenant deployments where different teams, users, or applications maintain separate semantic caches within the same infrastructure.
- Integrate 2D/3D cache cluster visualization using t-SNE or UMAP dimensionality reduction, allowing administrators to visually explore how cached queries group semantically and identify under-served topic areas.
- Extend the system to support streaming LLM responses with partial caching, enabling cache hits to serve the first N tokens instantly while the remaining tokens stream from the cloud LLM.
- Deploy as a SaaS product targeting AI startups and enterprise teams in India, offering a self-hosted or managed semantic caching layer that plugs into any existing LLM application with a single line of code.
- Incorporate query intent classification to route queries to specialized caches (e.g., factual queries vs. creative queries) with different similarity thresholds optimized for each intent type.

4. Tasks Accomplished

- Completed comprehensive literature review on semantic caching techniques, vector similarity search, dense embedding models, and LLM API cost optimization strategies — covering 5 key research papers and industry reports.
- Designed the complete system architecture as a four-service Docker Compose stack: FastAPI backend, PostgreSQL 16 + pgvector, Redis 7, and Next.js 15 frontend with clearly defined inter-service communication protocols.
- Defined the PostgreSQL database schema including the semantic_cache table with VECTOR(384) column, HNSW index configuration (m=16, ef_construction=64), query_log table for metrics tracking, and TTL-based cache expiry indexes.
- Evaluated and selected the embedding model (all-MiniLM-L6-v2) after benchmarking against all-mpnet-base-v2 and bge-small-en-v1.5 on dimensions of speed, VRAM usage, and semantic quality on paraphrase detection tasks.

- Designed the complete RESTful API specification with 10 endpoints covering query submission, cache management, metrics retrieval, settings configuration, and health checking — documented with OpenAPI/Swagger auto-generation via FastAPI.
- Established the project directory structure following Python best practices with modular separation: api/ (route handlers), core/ (business logic), db/ (database layer), and models/ (Pydantic schemas).
- Evaluated and selected the upstream LLM (Google Gemini 2.0 Flash) based on cost analysis (\$0.10/1M tokens vs. GPT-4o at \$2.50/1M tokens) and compatibility with the project's free-tier development constraints.
- Created the Docker Compose configuration with GPU passthrough for the embedding model, health checks for PostgreSQL, LRU eviction policy for Redis, and environment variable management via .env files.
- Designed the evaluation methodology including the 1,200-query benchmark corpus structure, threshold sensitivity analysis plan, and three-baseline comparison framework (no cache, exact-match, semantic cache).
- Prepared the project timeline with a 16-week development plan broken into phases: infrastructure (weeks 1–2), core logic (weeks 3–6), frontend (weeks 7–10), evaluation (weeks 11–12), and documentation (weeks 13–16).

5. Tasks in Progress / Next Plan

- Setting up the Docker Compose development environment with PostgreSQL 16 + pgvector, Redis 7, and FastAPI backend — targeting one-command startup with docker compose up --build.
- Implementing the embedding service wrapper class around SentenceTransformers with CUDA acceleration, batch encoding support, and vector normalization for cosine similarity.
- Building the core cache_router.py module implementing the two-tier lookup algorithm (Redis L1 → pgvector L2 → Cloud LLM fallback) with configurable similarity threshold.
- Developing the FastAPI endpoint handlers for POST /api/query (the main query route) and GET /api/metrics (dashboard data feed) with Pydantic request/response validation.
- Creating the seed_cache.py benchmarking script to generate the 1,200-query test corpus using paraphrase generation techniques for systematic evaluation.
- Beginning the Next.js 15 frontend scaffold with App Router, dark-mode Tailwind configuration, and the chat interface component for real-time query submission and cache hit/miss display.
- Writing the project report chapters: Introduction, Literature Review, System Design, and Implementation methodology.

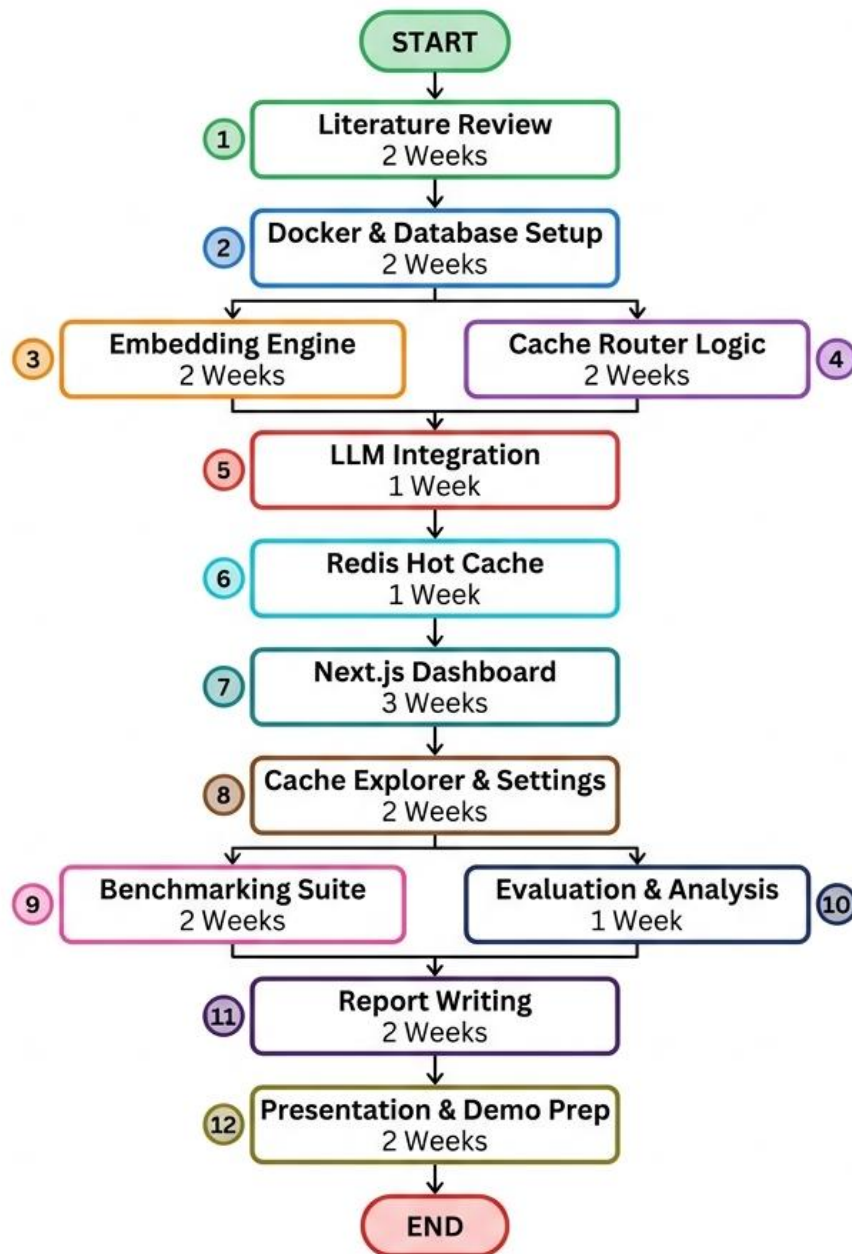
6. Learnings / Skills Acquired

- Deep understanding of dense vector embeddings and how transformer-based models (BERT, MiniLM) encode semantic meaning into high-dimensional vector spaces for similarity comparison.
- Hands-on experience with PostgreSQL's pgvector extension, including HNSW index tuning, cosine distance operators (<=>), and approximate nearest neighbor search performance characteristics.
- Proficiency in FastAPI development including async endpoint design, Pydantic schema validation, dependency injection, and automatic OpenAPI documentation generation.
- Practical knowledge of Docker Compose orchestration including multi-service networking, GPU passthrough configuration, health checks, and volume management for persistent data.
- Understanding of caching system design principles: two-tier cache hierarchies, TTL-based expiry, LRU eviction policies, cache invalidation strategies, and the trade-offs between hit rate and accuracy.
- Experience with LLM API integration and cost optimization — comparing pricing models across providers (Gemini, OpenAI, Anthropic) and understanding token-based billing structures.
- System design thinking: decomposing a complex middleware into modular, independently testable components (embedding service, cache router, LLM client, metrics logger) with clean interfaces.
- Research methodology skills: conducting literature surveys, identifying research gaps, designing evaluation benchmarks with precision/recall/F1 metrics, and performing threshold sensitivity analysis.



PERT CHART

CACHE-CRAFT SEMANTIC RAG ROUTER



Evaluation (To be filled by Mentor (Institute))

Criteria	Bloom's Level	Excellent (20)	Good (15)	Average (10)	Poor (5)	Marks Obtained (Out of 20)
Problem Definition	L4-Analyze	Clearly defined, justified real-world problem	Defined but lacks depth	Vague problem	No clear problem	
Project Title Relevance	L2-Understand	Highly precise and aligned	Mostly relevant	Partially relevant	Misleading	
Scope of Work	L5-Evaluate	Well-defined scope, deliverables, constraints	Defined with minor gaps	Broad/unclear	No scope	
Literature Study	L4-Analyze	3-5 sources analyzed critically	Sources listed	Minimal references	No study	
Clarity & Presentation	L3-Apply	Structured, professional	Minor issues	Some confusion	Poor	
Total Marks						

Evaluation

Student Signature	Mentor (Company) Signature	Mentor (Institute) Signature

Mentor (Institute) Remarks
